Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology, Guwahati

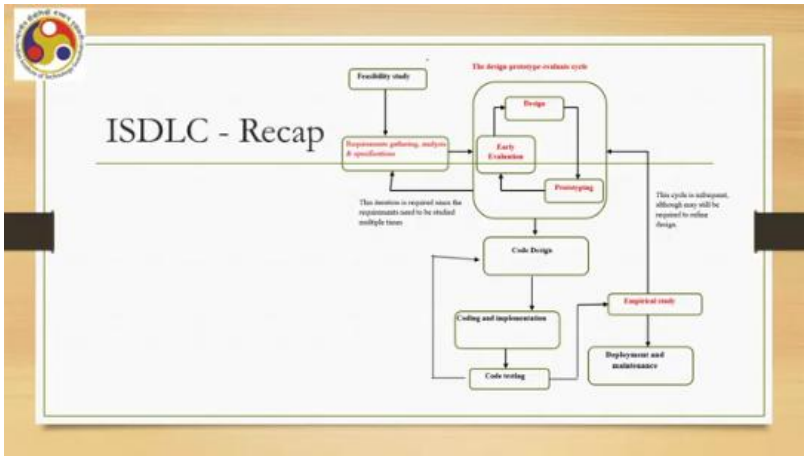
Week – 10

Lecture – 48

System Implementation

Hello and welcome to the NPTEL MOOCs course on Foundations for Virtual and Augmented Reality Systems. Today we are going to cover the remaining part of the implementation discussion that we are continuing. So, we will also continue the implementation discussion today, and after today's part, the discussion will come to an end. So, in order to understand this discussion, let us quickly recap. the interactive system development life cycle.

So, the life cycle is shown here. Now, we start with the feasibility study. Then we have requirement gathering, analysis, and specification. Then we have the design-prototype-evaluate cycle, followed by code design, coding and implementation, and code testing. followed by empirical study, followed by deployment and maintenance.



Now, out of these stages, we primarily talked about the requirement gathering stage, the design-prototype-evaluation stage, and the empirical study stage. So, in the design-prototype-evaluate cycle, there are three stages: design, prototyping, and quick evaluation of the prototype. So, we have covered these; other things we did not cover because these are standard, whereas the ones that we covered had some differences from other software development. So, we covered those in some detail. Now, the issue is, what is required?

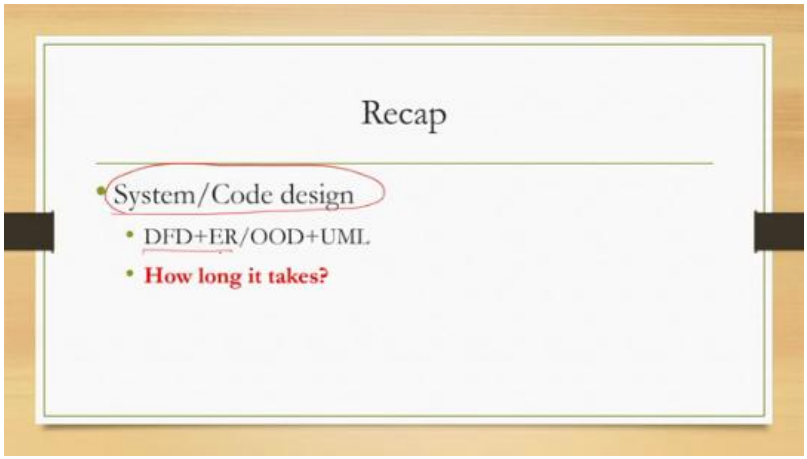
So, before we move to the issue, let us quickly recap what happens in each stage. So, in the requirement gathering stage, we gather functional and non-functional requirements, that is, usability requirements. We may need to conduct different methods or approaches to collect the requirements, such as contextual inquiry, which we have discussed in detail. Now, here two things are there.

One is the collection of requirements from end users and also clients. Now, sometimes they are the same; sometimes they are different. Most of the time, they are different. So, based on the requirements, we have to decide on functional and non-functional requirements. Usability requirements primarily come from end users. Some other non-functional requirements may come from clients, and functional requirements also may come from both clients and end users. Now, in order to identify all the requirements, all different types of requirements, it takes a long time. It may span months to collect all the possible requirements or all the requirements that are generally expected by both the end users as well as the clients. Then, once the requirements are gathered, we go to the interface design part, which comprises the design, prototype, and evaluate cycle.

Now, here in order to come up with a design, we may rely on intuition, or what we can do is start with design guidelines followed by brainstorming within the team to come up with a suitable design. This is followed by prototype building, followed by quick evaluation using methods such as cognitive walkthrough or heuristic evaluation, and this goes on in a cycle. Many times we may need to carry out the cycle so that eventually we come up with a stable interface design, meaning that further evaluation is not likely to throw up many new issues. How long can this stage take? It can also take months.

Although each cycle execution may take less time, one or two days, since we have to carry it out multiple times and it is not possible to fully devote time only for this purpose, there may be other activities. So, considering those, it may take between a couple of weeks to a month's time. Then comes the coding part. So, it starts with first the code design, which requires again

coming up with the design brainstorming like interface design; here the design language is DFD, ER, or object-oriented approaches, UML, and so on. Coming up with the proper design through several iterations and brainstorming again is time-consuming. It may also take months, or at least a few weeks, a couple of weeks. This is followed by implementation, coding, and testing.



Implementation involves coding and code testing. So, you need to first code, then test it. Primary level, secondary level; primary level may be review-based, secondary level may be a more systematic, structured way such as black box or white box testing. That can be done at the unit levels, then they are integrated, then system-level testing, and so on. So, this testing also is monotonous and time-consuming, and may take weeks.

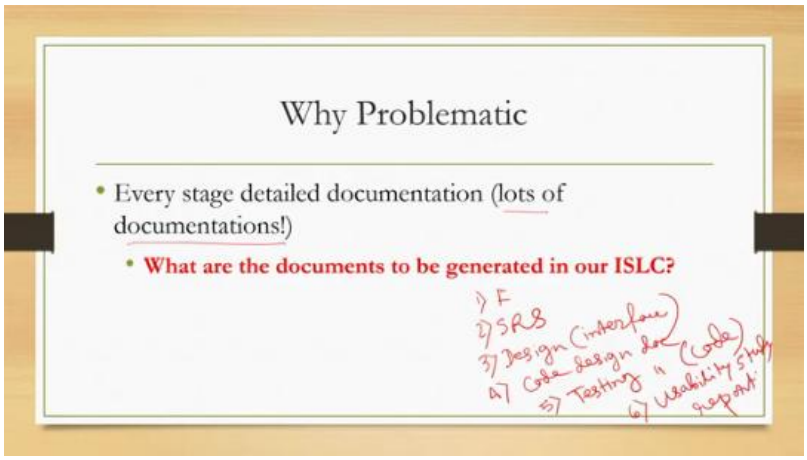
The last one is usability testing. That is, with code testing, we know that the system is executable, but we still do not know whether the system is usable. We expect the system to be usable because we have followed the design-prototype-evaluate cycle. However, whether that is true or there are still some gaps, we need to estimate through usability testing or an empirical study. Now, in an empirical study, we need to identify participants, we need to set up, and we need to get participants' time.

It may also take weeks. So, all these taken together take a long time. One problem with the traditional approach that we have discussed so far is its rigidity. That is, Every stage has to be completed before the next stage starts.

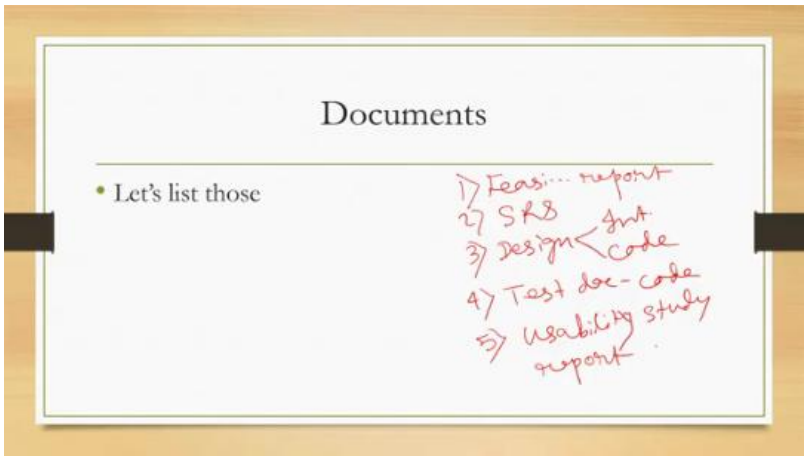
So, this may lead to time and cost overrun and even non-acceptance by the customer, that is the client, who may say that so much escalation in cost and time is not acceptable. Now, why is that problematic? This rigidity, because it has a strict requirement at each So, for example, at the requirement gathering stage, all requirements have to be identified first. Similarly, at the coding stage, all units have to be coded before they are sent to the testing phase. So, this type of outlook, the view that every stage has to be completed, creates the problem. So, what it says, the traditional way of doing things says, is that we need to implement or we need to carry out or we need to execute all activities and requirements at a stage before we can go to the next stage and before deployment. So when we talk of the deployment stage, that means everything is perfectly done: all requirements identified, coding done, code tested, usability tested, then only we go for deployment. Now, since each stage can take a long time, the eventual deployment may take a very, very long time. Also, another requirement is: why does each stage require a long time?

One is the execution of the stage; the other one is the need to keep documentation for each stage. So, there are lots of documents required in each stage. What are the documents? Say, for example, the feasibility study stage requires one. Then, the requirement gathering stage requires an SRS, a detailed SRS or software requirement specification documentation.

Then the design phase requires a design document. Now, this is about interface design. Code design also requires documentation so that it is easier to understand and debug by other teams. Testing documentation, code testing primarily, then a usability study report. So, a lot of documents need to be prepared if we follow the standard approach, the standard interactive system development lifecycle.

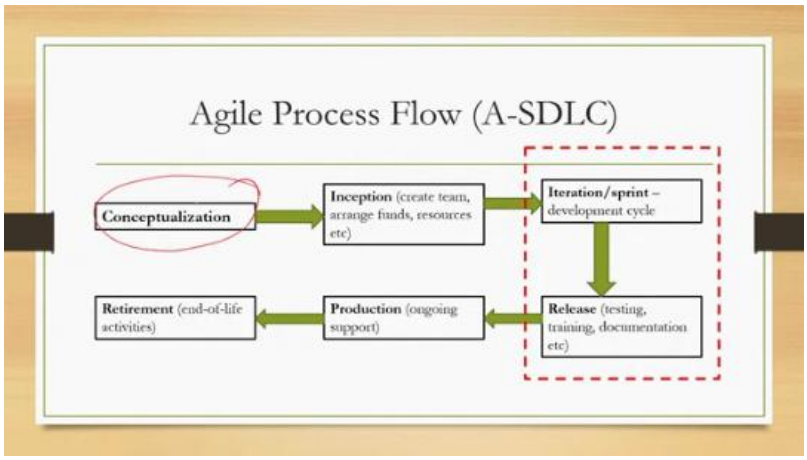


So, we have already listed such documents that are required. Just to recap, we need a feasibility study report. SRS is for the requirement gathering, analysis, and specification stage. SRS stands for software specification, software requirement specification, or system requirement specification. Then we need design documents both for interface and interaction, as well as code; then we have a testing document for code, and we also need a usability study report. Broadly, these five are required before we go for deployment. This is clearly problematic. It may lead to time and cost overruns. So, in order to overcome this situation, we need to adopt a method that can deliver quickly.



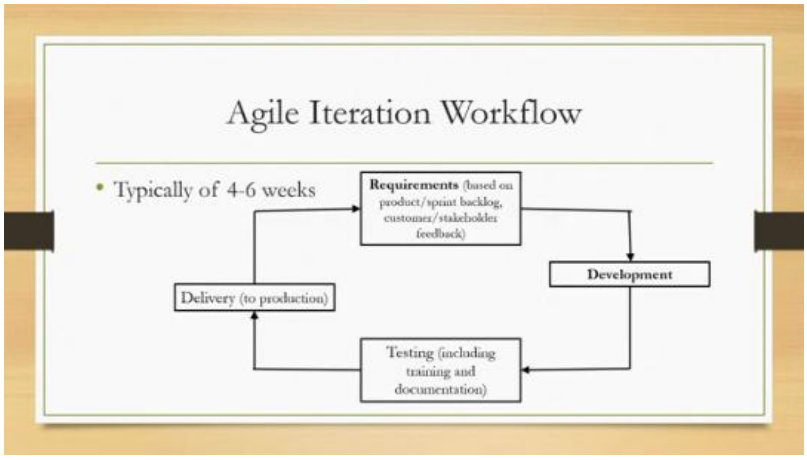
So there comes the agile development framework or agile development approach. The idea is that agile or fast development should be encouraged and should be implemented in practice. And if we implement this development practice, then it ensures quick turnaround of projects, which are more acceptable to the clients. So it eliminates unnecessary things and documentation. It starts small, small.

That means when this approach is adopted, we do not try to do everything from the very beginning. Not all requirements will be collected, nor will the entire system be developed and tested, and so on. So, what is this process flow? It is more or less similar to the standard system, shown in a different way, that is conceptualization. Then inception, which involves the creation of the team, arranging funds, resources, etc.



Then iteration, also known as sprint, which is the development cycle, followed by release, which includes testing, training, documentation, etc., followed by production, that is the ongoing support, followed by retirement, that is end-of-life activities. So production roughly refers to deployment and maintenance. Release refers to deployment, production refers to maintenance, and retirement is retirement. So conceptualization and inception roughly refer to both the physical feasibility study as well as requirement gathering.

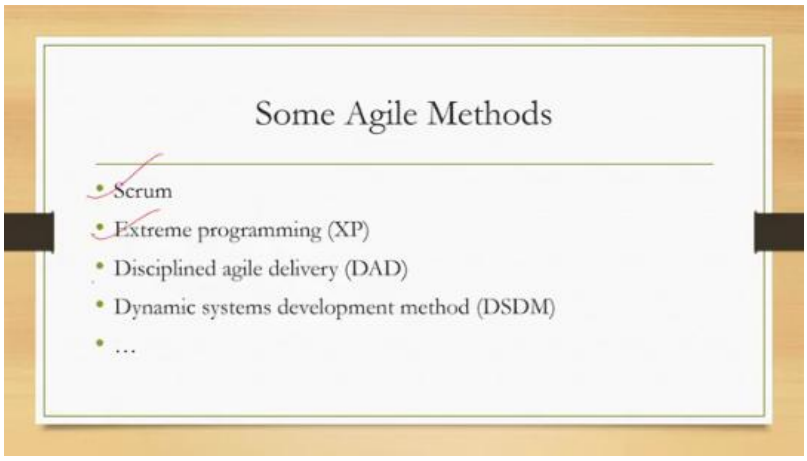
An iteration or sprint includes all the other stages, as we have seen in the interactive system development life cycle. Now, this iteration or sprint typically requires four to six weeks each, as compared to the whole ISDLC, the Interactive System Development Life Cycle, which may take months, six to nine months, whereas each iteration or sprint for development in the agile approach may take four to six weeks. And that is quite expected because we are not trying to do everything. So, this sprint includes requirements, development, testing, and delivery.



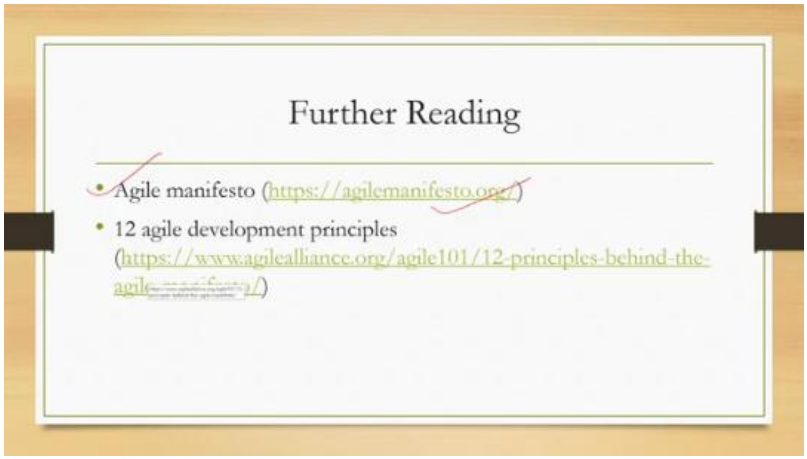
So, requirements based on the product sprint backlog and customer stakeholder feedback, then followed by development, then testing including training and documentation, followed by delivery to production. Four to six weeks is quite acceptable and manageable. So, how do we make it work? How can we deliver within four to six weeks?

It requires daily meetings or brainstorming by the development team, a live demo after each iteration to show the outcome of the iteration, and quick feedback regularly collected from stakeholders, customers, and shared with the team before the next sprint so that it can be addressed next time. The idea is to remain agile, that is, to be open to making changes based on feedback as quickly as possible. So, that is the overall idea, the overall philosophy.

We try to make things produced faster, even if that means that not everything is implemented; only some of the identified requirements are implemented. And as we keep on delivering, we keep on adding features, unlike the standard ISDLC method where we first complete the system and then deliver. So, there are many popular agile methods or frameworks available, such as SCRUM, Extreme Programming, DAD (Disciplined Agile Delivery), DSDM (Dynamic System Development Method), and so on. So, agile methods provide a better way to develop acceptable products, whereas ISDLC is primarily meant for understanding the overall flow rather than actually building practical systems within a reasonable time frame.



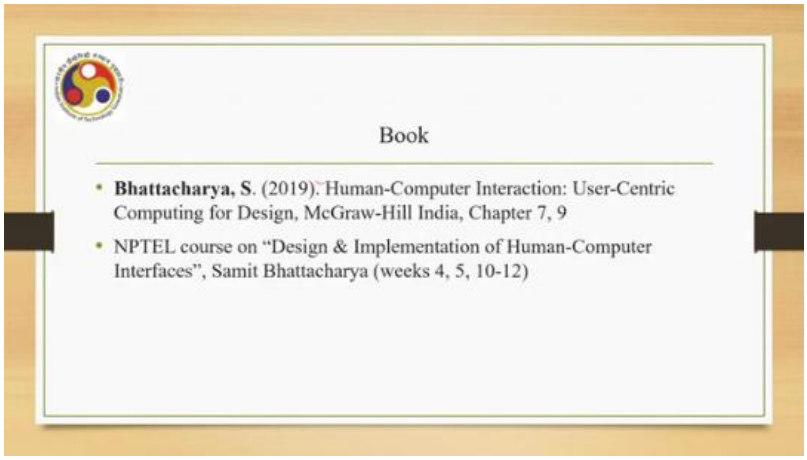
So, it is primarily for pedagogical purposes, whereas for practical purposes, agile frameworks are more suitable. To read more on this agile framework, you can refer to the agile manifesto, available online, and agile development principles, again available online. You can refer to these for further study. So, with that, we have come to the conclusion of this lecture.



So, what have we learned? We have talked about the prototype-evaluate part of the design-prototype-evaluate cycle: how to build and evaluate prototypes. Then we talked about empirical evaluation: how to finally evaluate usability, and also discussed the empirical evaluation using the case study of Majuli. And finally, we talked about the agile development method, which is the more practical way of building interactive systems as compared to the standard system development life cycles. So, this completes our discussion on

In the system implementation part earlier, we talked about design. First, we talked about requirement gathering, then design, followed by implementation. So, all three aspects of building a system are now complete. In the next few lectures, we will learn different aspects of virtual and augmented reality systems. Primarily, we will devote our attention to understanding an important topic: optimization. How do we optimize the experience as well as the system requirements? So, user experience optimization to manage psychophysiological constraints; we will talk about this.

Also, we will talk about adopting optimization approaches for resource or system optimization. So, for further studies on some of these concepts that we talked about here, such as prototype building, prototype evaluation, empirical study, as well as agile methods, you can refer to this book: Human-Computer Interaction, User-Centric Computing for Design. Also, you can refer to the NPTEL course on the design and implementation of human-computer interfaces, specifically weeks 4, 5, 10 to 12, for different discussions on the topics covered in this lecture.



So, with that, we come to the end of this lecture. I hope you could follow the topics, understand the concepts, and are in a position to apply the learning to practical scenarios. I look forward to meeting you all in the next lecture. Until then, thank you, goodbye, and happy learning.